ESP32 ULTRASONIC RADAR TRACKING SYSTEM

IoT Laboratory Course • Complete Project Source Code Dossier

- **Microcontroller:** ESP32 DevKit V1 (Xtensa 32-Bit Dual-Core @ 240 MHz)
- **Sensing & Actuation:** HC-SR04 Ultrasonic Distance Sensor + SG90 Micro Servo (0°–180°)
- **Network Architecture:** Wi-Fi 802.11 b/g/n • HTTP REST POST • Socket.IO WebSockets
- **Software Stack:** C++ (Arduino Core) • Node.js / Express • React 18 / Vite

---

1. EMBEDDED FIRMWARE: esp32/radar/radar.ino

C++ / Arduino IDE

```
/*
 * ESP32 Ultrasonic Radar
 *
 * Hardware:
 *   - ESP32 Development Board
 *   - HC-SR04 Ultrasonic Distance Sensor
 *   - SG90 (or MG90S) Micro Servo
 *
 * Wiring:
 *   HC-SR04:
 *     - VCC  → 5V (VIN on ESP32 or external 5V)
 *     - GND  → ESP32 GND
 *     - TRIG → GPIO 5
 *     - ECHO → GPIO 18 (USE VOLTAGE DIVIDER: 1kΩ from ECHO to GPIO18, 2kΩ from GPIO18 to GND)
 *
 *   SG90 Servo:
 *     - Signal (Orange/Yellow) → GPIO 19
 *     - V+ (Red)               → External regulated 5V supply (or VIN if USB provides enough current)
 *     - GND (Brown/Black)      → External 5V GND (MUST connect common ground with ESP32 GND)
 *
 *   Status LED:
 *     - GPIO 2 (Built-in blue LED on most ESP32 boards)
 *
 * Required Arduino Libraries:
 *   - ESP32Servo (by Kevin Harrington / John K. Bennett) - Install from Arduino IDE Library Manager
 *   - WiFi and HTTPClient (Included with ESP32 Arduino Board Package)
 */

#include <WiFi.h>
#include <HTTPClient.h>
#include <ESP32Servo.h>

// ═══════════════════ CONFIGURATION ═══════════════════
// 1. Enter your Wi-Fi credentials:
const char* WIFI_SSID     = "Pulin";
const char* WIFI_PASSWORD = "12344321";

// 2. Set this to the computer's current Wi-Fi IPv4 address (backend port 5000).
const char* SERVER_URL = "http://10.136.243.77:5000/radar";

// Pin definitions
#define TRIG_PIN       5
#define ECHO_PIN       18
#define SERVO_PIN      19
#define STATUS_LED_PIN 2

// Radar scanning parameters
const int   MIN_ANGLE       = 0;
const int   MAX_ANGLE       = 180;
const int   ANGLE_STEP      = 3;
```

```
const float MAX_DISTANCE_CM = 250.0;

// Settle and loop delays (in milliseconds)
const int SERVO_SETTLE_MS = 60;
const int LOOP_DELAY_MS   = 20;
// ============================================================

Servo radarServo;
WiFiClient wifiClient;

int angle = MIN_ANGLE;
int scanDirection = 1; // +1 for 0→180, -1 for 180→0
unsigned long lastWiFiCheck = 0;

// Measure distance using HC-SR04
float measureDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure echo pulse (30000us timeout = ~5.1m max)
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0) {
    return -1.0; // Timeout / no return echo
  }

  // Speed of sound: 343 m/s = 0.0343 cm/us
  float distance = (duration * 0.0343) / 2.0;

  if (distance < 2.0 || distance > MAX_DISTANCE_CM) {
    return -1.0; // Out of valid radar detection window
  }

  return distance;
}

// Connect to Wi-Fi network
void connectToWiFi() {
  Serial.println();
  Serial.print("[WiFi] Connecting to: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 25) {
    delay(400);
    digitalWrite(STATUS_LED_PIN, !digitalRead(STATUS_LED_PIN)); // Flash LED
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    digitalWrite(STATUS_LED_PIN, HIGH); // Solid ON when connected
    Serial.println();
    Serial.println("[WiFi] Connected successfully!");
    Serial.print("[WiFi] ESP32 IP Address: ");
    Serial.println(WiFi.localIP());
    Serial.print("[WiFi] Target Server: ");
    Serial.println(SERVER_URL);
  } else {
    digitalWrite(STATUS_LED_PIN, LOW);
    Serial.println();
    Serial.println("[WiFi] Warning: Could not connect to Wi-Fi. Operating in offline/serial mode.");
    Serial.println("[WiFi] Will automatically retry in background.");
  }
}
```

```
// Send radar reading to Node.js backend
bool sendRadarData(int currentAngle, float distance, bool detected) {
  if (WiFi.status() ≠ WL_CONNECTED) {
    return false;
  }

  HTTPClient http;
  http.begin(wifiClient, SERVER_URL);
  http.addHeader("Content-Type", "application/json");
  // Allow time for Wi-Fi connection setup and the backend response.
  http.setTimeout(1000);

  char payload[96];
  snprintf(payload, sizeof(payload),
    "{\"angle\":%d,\"distance\":%.1f,\"detected\":%s}",
    currentAngle,
    detected ? distance : 0.0,
    detected ? "true" : "false"
  );

  int httpCode = http.POST(payload);
  if (httpCode < 0) {
    Serial.print("[NET] POST failed: ");
    Serial.println(HTTPClient::errorToString(httpCode));
  } else if (httpCode ≠ HTTP_CODE_OK) {
    Serial.printf("[NET] Backend returned HTTP %d\n", httpCode);
  }
  http.end();

  return (httpCode ≥ 200 && httpCode < 300);
}

void setup() {
  Serial.begin(115200);
  delay(500);

  Serial.println();
  Serial.println("═══════════════════════════════════════════");
  Serial.println("     ESP32 ULTRASONIC RADAR STARTING      ");
  Serial.println("═══════════════════════════════════════════");

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(STATUS_LED_PIN, OUTPUT);
  digitalWrite(STATUS_LED_PIN, LOW);

  // Setup ESP32 Servo
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  ESP32PWM::allocateTimer(2);
  ESP32PWM::allocateTimer(3);
  radarServo.setPeriodHertz(50); // Standard 50Hz servo PWM
  radarServo.attach(SERVO_PIN, 500, 2400);

  // Move servo to starting position
  radarServo.write(MIN_ANGLE);
  delay(600);

  // Connect to Wi-Fi
  connectToWiFi();

  Serial.println("Radar scan loop initiated.");
  Serial.println("─────────────────────────────────────────");
}

void loop() {
  // Check Wi-Fi connection periodically
  if (millis() - lastWiFiCheck > 10000) {
    lastWiFiCheck = millis();
    if (WiFi.status() ≠ WL_CONNECTED) {
      digitalWrite(STATUS_LED_PIN, LOW);
```

```
      WiFi.reconnect();
    } else {
      digitalWrite(STATUS_LED_PIN, HIGH);
    }
  }

  // 1. Move servo to current angle
  radarServo.write(angle);

  // 2. Allow servo to reach target position
  delay(SERVO_SETTLE_MS);

  // 3. Measure ultrasonic distance
  float distance = measureDistanceCm();
  bool detected = (distance > 0.0);

  // 4. Send data to server over HTTP
  bool sent = sendRadarData(angle, distance, detected);

  // 5. Output to Serial Monitor
  Serial.printf("[RADAR] Angle: %3d deg | Dist: ", angle);
  if (detected) {
    Serial.printf("%5.1f cm", distance);
  } else {
    Serial.print("   ---  ");
  }
  Serial.printf(" | Status: %s | Net: %s\n",
    detected ? "OBJECT DETECTED" : "CLEAR",
    (WiFi.status() == WL_CONNECTED) ? (sent ? "SENT" : "FAIL") : "OFFLINE"
  );

  // 6. Step scanning angle
  angle += scanDirection * ANGLE_STEP;

  // Reverse sweep at limits
  if (angle ≥ MAX_ANGLE) {
    angle = MAX_ANGLE;
    scanDirection = -1;
  } else if (angle ≤ MIN_ANGLE) {
    angle = MIN_ANGLE;
    scanDirection = 1;
  }

  delay(LOOP_DELAY_MS);
}
```

2. RELAY GATEWAY BACKEND: backend/server.js Node.js / Express / Socket.IO

```
const express = require("express");
const http = require("http");
const cors = require("cors");
const os = require("os");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);

app.use(cors());
app.use(express.json({ limit: "10kb" }));

const io = new Server(server, {
  cors: { origin: "*" }
});

let latestReading = {
  angle: 0,
  distance: 0,
  detected: false,
  updatedAt: null,
  isMock: false
};

// Simulation / Mock State
let isMockActive = process.argv.includes("--mock");
let mockInterval = null;
let mockAngle = 0;
let mockDirection = 1;
const MOCK_STEP = 3;
const MOCK_SPEED_MS = 60;

function generateMockReading(angle) {
  // Simulate two objects in the field:
  // Target 1: at ~45 degrees, ~38cm away
  // Target 2: at ~125 degrees, ~85cm away
  let distance = 0;
  let detected = false;

  if (Math.abs(angle - 45) ≤ 6) {
    const jitter = (Math.random() - 0.5) * 1.5;
    distance = Math.round((38 + jitter) * 10) / 10;
    detected = true;
  } else if (Math.abs(angle - 125) ≤ 7) {
    const jitter = (Math.random() - 0.5) * 2.0;
    distance = Math.round((85 + jitter) * 10) / 10;
    detected = true;
  }

  return {
    angle,
    distance,
    detected,
    updatedAt: new Date().toISOString(),
    isMock: true
  };
}

function startMock() {
  if (mockInterval) return;
  isMockActive = true;
  console.log(">> Radar Mock/Simulation MODE ACTIVE <<");

  mockInterval = setInterval(() ⇒ {
    latestReading = generateMockReading(mockAngle);
    io.emit("radar-reading", latestReading);
```

```
    mockAngle += mockDirection * MOCK_STEP;
    if (mockAngle ≥ 180) {
      mockAngle = 180;
      mockDirection = -1;
    } else if (mockAngle ≤ 0) {
      mockAngle = 0;
      mockDirection = 1;
    }
  }, MOCK_SPEED_MS);

  io.emit("mock-status", { active: true });
}

function stopMock() {
  if (!mockInterval) return;
  clearInterval(mockInterval);
  mockInterval = null;
  isMockActive = false;
  console.log(">> Radar Mock/Simulation MODE STOPPED <<");
  io.emit("mock-status", { active: false });
}

if (isMockActive) {
  startMock();
}

app.get("/health", (_req, res) ⇒ {
  res.json({
    status: "ok",
    service: "ESP32 ultrasonic radar backend",
    isMockActive,
    latestReading
  });
});

app.get("/api/latest", (_req, res) ⇒ {
  res.json(latestReading);
});

app.get("/api/mock/status", (_req, res) ⇒ {
  res.json({ active: isMockActive });
});

app.post("/api/mock/toggle", (_req, res) ⇒ {
  if (isMockActive) {
    stopMock();
  } else {
    startMock();
  }
  res.json({ active: isMockActive });
});

app.post("/radar", (req, res) ⇒ {
  const { angle, distance, detected } = req.body || {};

  if (
    typeof angle ≠≡ "number" || angle < 0 || angle > 180 ||
    typeof distance ≠≡ "number" || !Number.isFinite(distance) ||
    typeof detected ≠≡ "boolean"
  ) {
    return res.status(400).json({
      error: "Expected angle (0-180), distance (number), detected (boolean)"
    });
  }

  // If real data arrives from ESP32, automatically pause mock if it was running
  if (isMockActive && !req.query.allowMock) {
    console.log("Real ESP32 data received. Suspending mock mode.");
    stopMock();
  }

  latestReading = {
```

```
    angle: Math.round(angle),
    distance: Math.round(distance * 10) / 10,
    detected,
    updatedAt: new Date().toISOString(),
    isMock: false
  };

  io.emit("radar-reading", latestReading);
  res.status(200).json({ received: true });
});

io.on("connection", (socket) => {
  console.log(`[Socket] Dashboard connected: ${socket.id}`);
  socket.emit("radar-reading", latestReading);
  socket.emit("mock-status", { active: isMockActive });

  socket.on("toggle-mock", () => {
    if (isMockActive) {
      stopMock();
    } else {
      startMock();
    }
  });

  socket.on("disconnect", () => {
    console.log(`[Socket] Dashboard disconnected: ${socket.id}`);
  });
});

function getLocalIpAddresses() {
  const nets = os.networkInterfaces();
  const ips = [];
  for (const name of Object.keys(nets)) {
    for (const net of nets[name]) {
      if (net.family === "IPv4" && !net.internal) {
        ips.push({ interface: name, address: net.address });
      }
    }
  }
  return ips;
}

const PORT = process.env.PORT || 5000;
server.listen(PORT, "0.0.0.0", () => {
  console.log("\n═══════════════════════════════════════════════════════");
  console.log(`  RADAR BACKEND SERVER RUNNING ON PORT ${PORT}`);
  console.log("═══════════════════════════════════════════════════════");
  console.log(`Local Access:   http://localhost:${PORT}`);

  const localIps = getLocalIpAddresses();
  if (localIps.length > 0) {
    console.log("Available Network Interfaces (for ESP32 and mobile/tablet):");
    localIps.forEach((item) => {
      console.log(`  - [${item.interface}] http://${item.address}:${PORT}`);
      console.log(`    ESP32 SERVER_URL → http://${item.address}:${PORT}/radar`);
    });
  }
  console.log(`Simulation Mode: ${isMockActive ? "ACTIVE" : "OFF (use 'npm run mock' to activate)"}`);
  console.log("═══════════════════════════════════════════════════════\n");
});
```

3. BACKEND DEPENDENCIES: backend/package.json JSON

```
{
  "name": "esp32-ultrasonic-radar-backend",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "scripts": {
```

```
    "start": "node server.js",
    "mock": "node server.js --mock",
    "dev": "node --watch server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.21.2",
    "socket.io": "^4.8.1"
  }
}
```

4. RADAR HUD COMPONENT: frontend/src/App.jsx React 18 / JSX

```
import React, { useEffect, useMemo, useRef, useState } from "react";
import { io } from "socket.io-client";

// Resolve backend server address dynamically:
// 1. Vite environment variable (if specified via .env or VITE_SERVER_URL)
// 2. Or automatically match the current host with port 5000 (localhost or LAN IP)
const getDefaultServerUrl = () => {
  if (import.meta.env.VITE_SERVER_URL) {
    return import.meta.env.VITE_SERVER_URL;
  }
  if (typeof window !== "undefined" && window.location.hostname) {
    return `${window.location.protocol}//${window.location.hostname}:5000`;
  }
  return "http://localhost:5000";
};

const MAX_RANGE = 250; // Maximum distance in cm
const HISTORY_LIMIT = 180;

function playSonarPing() {
  try {
    const AudioCtx = window.AudioContext || window.webkitAudioContext;
    if (!AudioCtx) return;
    const ctx = new AudioCtx();
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();

    osc.type = "sine";
    osc.frequency.setValueAtTime(880, ctx.currentTime);
    osc.frequency.exponentialRampToValueAtTime(440, ctx.currentTime + 0.12);

    gain.gain.setValueAtTime(0.15, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.12);

    osc.connect(gain);
    gain.connect(ctx.destination);

    osc.start();
    osc.stop(ctx.currentTime + 0.13);
  } catch {
    // Ignore audio autoplay restrictions
  }
}

function RadarCanvas({ readings, currentAngle, sweepDirection, maxRange }) {
  const width = 760;
  const height = 410;
  const cx = width / 2;
  const cy = height - 26;
  const radius = 340;

  // Polar projection: 0° is left, 90° is top, 180° is right
  const pointFor = (angle, distance) => {
    const radians = ((180 - angle) * Math.PI) / 180;
    const scaled = (Math.min(distance, maxRange) / maxRange) * radius;
    return {
      x: cx + Math.cos(radians) * scaled,
      y: cy - Math.sin(radians) * scaled
    };
  };

  const arcPath = (r) =>
    `M ${cx - r} ${cy} A ${r} ${r} 0 0 1 ${cx + r} ${cy}`;

  const sweep = pointFor(currentAngle, radius);

  // Compute realistic trailing phosphor sweep sector
```

```
  const trailingSpan = 16;
  const trailingAngle =
    sweepDirection ≥ 0
      ? Math.max(0, currentAngle - trailingSpan)
      : Math.min(180, currentAngle + trailingSpan);

  const minA = Math.min(trailingAngle, currentAngle);
  const maxA = Math.max(trailingAngle, currentAngle);
  const pStart = pointFor(minA, radius);
  const pEnd = pointFor(maxA, radius);
  const sweepWedge = `M ${cx} ${cy} L ${pStart.x} ${pStart.y} A ${radius} ${radius} 0 0 1 ${pEnd.x} ${pEnd.y} Z
`;

  return (
    <svg
      className="radar-svg"
      viewBox={`0 0 ${width} ${height}`}
      role="img"
      aria-label="Live semicircular ultrasonic radar"
    >
      <defs>
        <radialGradient id="sweepGlow" cx="50%" cy="100%" r="90%">
          <stop offset="0%" stopColor="var(--accent)" stopOpacity="0.08" />
          <stop offset="60%" stopColor="var(--accent)" stopOpacity="0.28" />
          <stop offset="100%" stopColor="var(--accent)" stopOpacity="0.5" />
        </radialGradient>
      </defs>

      {/* Concentric Range Rings */}
      {[0.25, 0.5, 0.75, 1].map((fraction) => (
        <g key={fraction}>
          <path d={arcPath(radius * fraction)} className="range-arc" />
          <text
            x={cx + 8}
            y={cy - radius * fraction + 4}
            className="range-label"
          >
            {Math.round(maxRange * fraction)} cm
          </text>
        </g>
      ))}

      {/* Bearing Guide Lines */}
      {[0, 30, 60, 90, 120, 150, 180].map((angle) => {
        const p = pointFor(angle, maxRange);
        const label = pointFor(angle, maxRange + 16);
        return (
          <g key={angle}>
            <line x1={cx} y1={cy} x2={p.x} y2={p.y} className="bearing-line" />
            <text
              x={label.x}
              y={label.y}
              textAnchor="middle"
              className="bearing-label"
            >
              {angle}°
            </text>
          </g>
        );
      })}

      {/* Trailing Radar Sweep Sector */}
      {minA ≠= maxA && (
        <path d={sweepWedge} fill="url(#sweepGlow)" opacity="0.65" />
      )}

      {/* Active Sweep Beam Line */}
      <line
        x1={cx}
        y1={cy}
        x2={sweep.x}
        y2={sweep.y}
```

```
        className="sweep-line"
      />

      {/* Detected Target Blips with Phosphor Decay */}
      {readings
        .map((item, index) => ({ item, index }))
        .filter(({ item }) => item.detected && item.distance > 0)
        .map(({ item, index }) => {
          const p = pointFor(item.angle, item.distance);
          const opacity = Math.min(1, Math.max(0.25, (index + 1) / readings.length));
          const isCritical = item.distance <= 35;

          return (
            <g key={`${item.angle}-${index}`} opacity={opacity}>
              {isCritical && (
                <circle
                  cx={p.x}
                  cy={p.y}
                  r="9"
                  className="target-glow-ring"
                />
              )}
              <circle
                cx={p.x}
                cy={p.y}
                r={isCritical ? "6" : "4.5"}
                className={isCritical ? "target-dot critical" : "target-dot"}
              />
            </g>
          );
        })}

      {/* Origin Center Sensor Dot */}
      <circle cx={cx} cy={cy} r="7" className="origin-dot" />
      <text x={cx} y={cy + 20} textAnchor="middle" className="origin-label">
        HC-SR04 SENSOR
      </text>
    </svg>
  );
}

export default function App() {
  const [theme, setTheme] = useState(
    () => localStorage.getItem("radar-theme") || "dark"
  );
  const [serverUrl, setServerUrl] = useState(getDefaultServerUrl);
  const [showConfig, setShowConfig] = useState(false);
  const [connection, setConnection] = useState("connecting");
  const [isMocking, setIsMocking] = useState(false);
  const [audioEnabled, setAudioEnabled] = useState(false);

  const [latest, setLatest] = useState({
    angle: 0,
    distance: 0,
    detected: false,
    updatedAt: null,
    isMock: false
  });
  const [readings, setReadings] = useState([]);
  const [sweepDirection, setSweepDirection] = useState(1);
  const prevAngleRef = useRef(0);
  const socketRef = useRef(null);

  useEffect(() => {
    localStorage.setItem("radar-theme", theme);
  }, [theme]);

  // Connect to backend via Socket.IO
  useEffect(() => {
    const socket = io(serverUrl, {
      reconnection: true,
      reconnectionAttempts: Infinity,
```

```
      reconnectionDelay: 1000,
      timeout: 5000
    });
    socketRef.current = socket;

    socket.on("connect", () => {
      setConnection("online");
    });

    socket.on("disconnect", () => {
      setConnection("offline");
    });

    socket.on("connect_error", () => {
      setConnection("offline");
    });

    socket.on("mock-status", (status) => {
      setIsMocking(Boolean(status?.active));
    });

    socket.on("radar-reading", (reading) => {
      if (!reading || typeof reading.angle !== "number") return;

      // Determine sweep direction (+1 forward, -1 backward)
      if (reading.angle > prevAngleRef.current) {
        setSweepDirection(1);
      } else if (reading.angle < prevAngleRef.current) {
        setSweepDirection(-1);
      }
      prevAngleRef.current = reading.angle;

      setLatest(reading);
      if (reading.isMock !== undefined) {
        setIsMocking(reading.isMock);
      }

      setReadings((previous) => [...previous, reading].slice(-HISTORY_LIMIT));

      // Sonar ping when an object is detected
      if (reading.detected && audioEnabled) {
        playSonarPing();
      }
    });

    return () => {
      socket.disconnect();
    };
  }, [serverUrl, audioEnabled]);

  const toggleMock = () => {
    if (socketRef.current && connection === "online") {
      socketRef.current.emit("toggle-mock");
    } else {
      // Direct REST fallback
      fetch(`${serverUrl}/api/mock/toggle`, { method: "POST" })
        .then((res) => res.json())
        .then((data) => setIsMocking(data.active))
        .catch(() => alert("Could not reach backend server at " + serverUrl));
    }
  };

  const detectedCount = useMemo(
    () => readings.filter((item) => item.detected).length,
    [readings]
  );

  const proximityWarning = latest.detected && latest.distance > 0 && latest.distance <= 35;

  return (
    <main className="app-shell" data-theme={theme}>
      <header className="topbar">
```

```
        <div className="brand">
          <div className="brand-mark">R</div>
          <div>
            <p className="eyebrow">FIELD SYSTEMS / IOT RADAR</p>
            <h1>ULTRASONIC RADAR MONITOR</h1>
          </div>
        </div>

        <div className="header-actions">
          {/* Simulation Toggle Button */}
          <button
            className={`sim-button ${isMocking ? "active" : ""}`}
            onClick={toggleMock}
            title="Toggle simulated test targets (great for presentations without hardware)"
          >
            <span className="sim-dot" />
            {isMocking ? "SIMULATOR: ACTIVE" : "SIMULATE"}
          </button>

          {/* Audio Beep Toggle */}
          <button
            className={`audio-button ${audioEnabled ? "active" : ""}`}
            onClick={() => setAudioEnabled((v) => !v)}
            title="Sonar audio chime on target detection"
          >
            {audioEnabled ? "SONAR: ON" : "SONAR: MUTE"}
          </button>

          {/* Connection Pill */}
          <div
            className={`connection ${connection}`}
            onClick={() => setShowConfig((v) => !v)}
            title="Click to view/change backend URL"
            style={{ cursor: "pointer" }}
          >
            <span className="status-light" />
            {connection === "online"
              ? isMocking
                ? "LINK: SIMULATOR"
                : "LINK: ACTIVE"
              : connection === "connecting"
              ? "LINKING..."
              : "LINK: OFFLINE"}
          </div>

          {/* Theme Button */}
          <button
            className="theme-button"
            onClick={() =>
              setTheme((value) => (value === "dark" ? "light" : "dark"))
            }
          >
            {theme === "dark" ? "LIGHT" : "DARK"}
          </button>
        </div>
      </header>

      {/* Backend URL Configuration Drawer */}
      {showConfig && (
        <div className="config-banner">
          <div className="config-inner">
            <label htmlFor="backend-url-input">Backend Server URL:</label>
            <input
              id="backend-url-input"
              type="text"
              value={serverUrl}
              onChange={(e) => setServerUrl(e.target.value)}
              placeholder="http://10.20.5.186:5000"
            />
            <button
              onClick={() => {
                setShowConfig(false);
```

```
            }}
          >
            SAVE
          </button>
          <span className="config-hint">
            Tip: Set this to your computer's IP address if opening on another device.
          </span>
        </div>
      </div>
    )}

    {/* Status Bar */}
    <section className="status-strip">
      <span>SCAN RANGE: 0°–180°</span>
      <span>MAX DETECTION: {MAX_RANGE} CM</span>
      <span>RESOLUTION: 3° STEP</span>
      <span>
        SOURCE:{" "}
        <strong style={{ color: isMocking ? "var(--danger)" : "var(--accent)" }}>
          {isMocking ? "SYNTHETIC SIMULATION" : "ESP32 HARDWARE"}
        </strong>
      </span>
    </section>

    {/* Main Grid */}
    <section className="main-grid">
      <article className="panel radar-panel">
        <div className="panel-heading">
          <div>
            <p className="eyebrow">POLAR RADAR PROJECTION</p>
            <h2>LIVE SWEEP DISPLAY</h2>
          </div>
          <div className="heading-badges">
            {proximityWarning && (
              <span className="proximity-badge">PROXIMITY ALERT</span>
            )}
            <span className="live-tag">
              <span /> LIVE
            </span>
          </div>
        </div>

        <div className="radar-wrap">
          <RadarCanvas
            readings={readings}
            currentAngle={latest.angle}
            sweepDirection={sweepDirection}
            maxRange={MAX_RANGE}
          />
        </div>

        <div className="scale-row">
          <span>0° / LEFT</span>
          <span>90° / CENTER</span>
          <span>180° / RIGHT</span>
        </div>
      </article>

      <aside className="side-column">
        {/* Bearing & Range Panel */}
        <article className="panel metric-panel">
          <p className="eyebrow">SERVO BEARING</p>
          <div className="big-value">
            {latest.angle}
            <small>°</small>
          </div>
          <div className="metric-rule" />
          <p className="eyebrow">MEASURED RANGE</p>
          <div className="big-value">
            {latest.detected ? latest.distance.toFixed(1) : "–"}
            <small>{latest.detected ? "cm" : "cm"}</small>
          </div>
```

```
            {latest.detected && (
              <p className="small-note" style={{ marginTop: "6px" }}>
                ≈ {(latest.distance / 2.54).toFixed(1)} inches
              </p>
            )}
          </article>

          {/* Detection Status Panel */}
          <article className="panel detection-panel">
            <p className="eyebrow">TARGET CLASSIFICATION</p>
            <div
              className={`target-state ${
                proximityWarning
                  ? "critical"
                  : latest.detected
                  ? "detected"
                  : "clear"
              }`}
            >
              <span className="target-indicator" />
              {proximityWarning
                ? "CRITICAL PROXIMITY"
                : latest.detected
                ? "TARGET ECHO DETECTED"
                : "RADAR HORIZON CLEAR"}
            </div>
            <p className="small-note">
              {proximityWarning
                ? "Obstacle within critical 35 cm zone!"
                : latest.detected
                ? `Ultrasonic echo returned at ${latest.angle}° bearing (${latest.distance.toFixed(1)} cm).`
                : "No obstacle echo returned within 250 cm limit."}
            </p>
          </article>

          {/* Session Data */}
          <article className="panel log-panel">
            <p className="eyebrow">TELEMETRY STATS</p>
            <div className="log-row">
              <span>Data Link</span>
              <strong>{connection.toUpperCase()}</strong>
            </div>
            <div className="log-row">
              <span>Samples Logged</span>
              <strong>{readings.length}</strong>
            </div>
            <div className="log-row">
              <span>Target Returns</span>
              <strong>{detectedCount}</strong>
            </div>
            <div className="log-row">
              <span>Last Packet</span>
              <strong>
                {latest.updatedAt
                  ? new Date(latest.updatedAt).toLocaleTimeString()
                  : "WAITING FOR DATA"}
              </strong>
            </div>
          </article>
        </aside>
      </section>

      <footer className="footer">
        <span>ESP32 (GPIO 5 / 18 / 19) • HC-SR04 • SG90 SERVO</span>
        <span>STATUS: {isMocking ? "SIMULATION BENCH" : "OPERATIONAL"}</span>
      </footer>
    </main>
  );
}
```

5. RADAR HUD STYLING: frontend/src/App.css CSS3 (Tactical HUD)

```
* { box-sizing: border-box; }
:root {
  font-family: "Segoe UI", -apple-system, BlinkMacSystemFont, "Roboto", sans-serif;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}
body { margin: 0; min-width: 320px; }
button { font: inherit; }

.app-shell {
  --bg: #07110c;
  --panel: #0c1b12;
  --panel-2: #102419;
  --text: #e5f2e8;
  --muted: #829b88;
  --line: #1e3b29;
  --accent: #69e58b;
  --accent-strong: #2bc75b;
  --danger: #ffb35c;
  --critical: #ff4757;
  min-height: 100vh;
  padding: 24px clamp(16px, 4vw, 48px) 20px;
  color: var(--text);
  background:
    radial-gradient(ellipse at 50% 0%, rgba(39, 117, 62, .14), transparent 48%),
    var(--bg);
  transition: background .2s ease, color .2s ease;
}

.app-shell[data-theme="light"] {
  --bg: #eef5ef;
  --panel: #ffffff;
  --panel-2: #f5faf5;
  --text: #173321;
  --muted: #67816e;
  --line: #cbdccd;
  --accent: #168c42;
  --accent-strong: #087b34;
  --danger: #d97706;
  --critical: #dc2626;
  background: #eef5ef;
}

.topbar, .brand, .header-actions, .panel-heading, .status-strip, .scale-row,
.footer, .log-row { display: flex; align-items: center; justify-content: space-between; }

.topbar { gap: 20px; padding-bottom: 20px; border-bottom: 1px solid var(--line); }
.brand { justify-content: flex-start; gap: 14px; }
.brand-mark {
  width: 44px; height: 44px; display: grid; place-items: center;
  border: 1.5px solid var(--accent); color: var(--accent);
  font-weight: 900; font-size: 24px; transform: skew(-5deg);
  background: rgba(105, 229, 139, 0.05);
}
.eyebrow { margin: 0 0 6px; color: var(--muted); font-size: 10px; font-weight: 800; letter-spacing: 2px; }
h1 { margin: 0; font-size: clamp(18px, 2.2vw, 24px); letter-spacing: 1.5px; }

.header-actions { gap: 10px; flex-wrap: wrap; }

.connection, .theme-button, .sim-button, .audio-button {
  border: 1px solid var(--line); background: var(--panel); color: var(--text);
  padding: 9px 12px; font-size: 10px; font-weight: 800; letter-spacing: 1px;
  cursor: pointer; transition: all .15s ease;
}
.connection { display: flex; align-items: center; gap: 8px; cursor: pointer; }
.connection:hover { border-color: var(--accent); }
```

```
.status-light, .live-tag span, .sim-dot {
  display: inline-block; width: 8px; height: 8px; border-radius: 50%; background: var(--muted);
}
.connection.online .status-light, .live-tag span { background: var(--accent); box-shadow: 0 0 10px var(--accen
t); }
.connection.offline .status-light { background: var(--critical); box-shadow: 0 0 10px var(--critical); }
.connection.connecting .status-light { background: var(--danger); }

.sim-button { display: flex; align-items: center; gap: 8px; }
.sim-button.active {
  border-color: var(--danger);
  color: var(--danger);
  background: rgba(255, 179, 92, 0.1);
}
.sim-button.active .sim-dot {
  background: var(--danger);
  box-shadow: 0 0 10px var(--danger);
}

.audio-button.active {
  border-color: var(--accent);
  color: var(--accent);
}

.theme-button:hover, .audio-button:hover { border-color: var(--accent); color: var(--accent); }

.config-banner {
  background: var(--panel);
  border: 1px solid var(--accent);
  padding: 12px 18px;
  margin-top: 14px;
}
.config-inner {
  display: flex;
  align-items: center;
  gap: 12px;
  flex-wrap: wrap;
  font-size: 12px;
}
.config-inner input {
  background: var(--bg);
  border: 1px solid var(--line);
  color: var(--text);
  padding: 6px 10px;
  font-size: 12px;
  min-width: 280px;
}
.config-inner button {
  background: var(--accent);
  color: var(--bg);
  border: none;
  font-weight: 800;
  padding: 7px 16px;
  cursor: pointer;
}
.config-hint { color: var(--muted); font-size: 11px; }

.status-strip {
  justify-content: flex-start; flex-wrap: wrap; gap: 24px; padding: 12px 0;
  color: var(--muted); font-size: 10px; letter-spacing: 1.4px; border-bottom: 1px solid var(--line);
}

.main-grid { display: grid; grid-template-columns: minmax(0, 1.8fr) minmax(250px, .8fr); gap: 16px; margin-top:
16px; }
.panel { background: var(--panel); border: 1px solid var(--line); }
.radar-panel { padding: 18px 18px 12px; min-width: 0; }

.panel-heading { gap: 10px; }
.panel-heading h2 { margin: 0; font-size: 14px; letter-spacing: 1.6px; }

.heading-badges { display: flex; align-items: center; gap: 12px; }
.proximity-badge {
```

```
  color: var(--critical);
  background: rgba(255, 71, 87, 0.15);
  border: 1px solid var(--critical);
  padding: 4px 8px;
  font-size: 9px;
  font-weight: 900;
  letter-spacing: 1px;
  animation: pulse-crit 1s infinite alternate;
}
@keyframes pulse-crit {
  from { opacity: 0.7; }
  to { opacity: 1; box-shadow: 0 0 10px var(--critical); }
}

.live-tag { color: var(--accent); display: flex; align-items: center; gap: 8px; font-size: 10px; font-weight: 9
00; letter-spacing: 1px; }

.radar-wrap { width: 100%; margin-top: 12px; overflow: hidden; }
.radar-svg { width: 100%; display: block; }
.range-arc { fill: none; stroke: var(--line); stroke-width: 1.2; }
.bearing-line { stroke: var(--line); stroke-width: 1; stroke-dasharray: 3 5; }
.range-label, .bearing-label, .origin-label {
  fill: var(--muted); font-size: 10px; font-family: inherit;
}
.sweep-line {
  stroke: var(--accent);
  stroke-width: 2.2;
  filter: drop-shadow(0 0 6px var(--accent));
}

.target-dot {
  fill: #ffb35c;
  stroke: #fff0d9;
  stroke-width: 1;
  filter: drop-shadow(0 0 6px #ffb35c);
  transition: opacity .3s ease;
}
.target-dot.critical {
  fill: #ff4757;
  stroke: #ffffff;
  filter: drop-shadow(0 0 8px #ff4757);
}
.target-glow-ring {
  fill: none;
  stroke: #ff4757;
  stroke-width: 1.5;
  stroke-dasharray: 2 2;
  opacity: 0.8;
  animation: spin-glow 3s linear infinite;
}

.origin-dot { fill: var(--accent); }
.origin-label { font-size: 8px; letter-spacing: 1px; font-weight: bold; }
.scale-row { color: var(--muted); font-size: 9px; letter-spacing: 1px; padding: 2px 8px 4px; }

.side-column { display: flex; flex-direction: column; gap: 16px; }
.metric-panel, .detection-panel, .log-panel { padding: 18px; }
.big-value { font-size: clamp(38px, 4.8vw, 52px); line-height: 1; font-weight: 300; letter-spacing: -2px; }
.big-value small { font-size: 14px; color: var(--muted); letter-spacing: 0; margin-left: 6px; }
.metric-rule { height: 1px; background: var(--line); margin: 18px 0; }

.target-state { display: flex; align-items: center; gap: 10px; font-size: 12px; font-weight: 900; letter-spacin
g: .8px; }
.target-indicator { width: 10px; height: 10px; border-radius: 50%; background: var(--muted); }
.target-state.detected { color: var(--danger); }
.target-state.detected .target-indicator { background: var(--danger); box-shadow: 0 0 10px var(--danger); }
.target-state.critical { color: var(--critical); }
.target-state.critical .target-indicator { background: var(--critical); box-shadow: 0 0 12px var(--critical); }
.target-state.clear { color: var(--muted); }

.small-note { color: var(--muted); font-size: 11px; line-height: 1.5; margin: 12px 0 0; }
.log-row { padding: 9px 0; border-bottom: 1px solid var(--line); font-size: 11px; }
```

```
.log-row:last-child { border: 0; }
.log-row span { color: var(--muted); }
.log-row strong { font-size: 11px; }

.footer { margin-top: 20px; padding-top: 14px; border-top: 1px solid var(--line); color: var(--muted); font-siz
e: 9px; letter-spacing: 1.3px; }

@media (max-width: 850px) {
  .main-grid { grid-template-columns: 1fr; }
  .side-column { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); }
  .log-panel { grid-column: 1 / -1; }
}
@media (max-width: 560px) {
  .topbar { align-items: flex-start; flex-direction: column; }
  .header-actions { width: 100%; justify-content: space-between; }
  .side-column { grid-template-columns: 1fr; }
  .log-panel { grid-column: auto; }
  .status-strip { gap: 12px; }
  .scale-row { font-size: 7px; }
}
```

6. BUILD CONFIGURATION: frontend/vite.config.js Vite / ESModule

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    host: "0.0.0.0",
    port: 5173
  }
});
```